День 4. Мониторинг базы данных MySQL с помощью Zabbix-агента. Написание простого скрипта

Этот раздел посвящен технологиям мониторинга баз данных. В качестве объекта наблюдения будет использована СУБД MySQL, обслуживающая систему мониторинга Zabbix. База данных, по сути - совокупность упорядоченных данных, которые могут использовать приложения для работы с ними.

Мониторинг баз данных с использованием Zabbix можно осуществлять несколькими способами:

- Внешняя проверка (запуск скрипта, написанного администратором):

Преимущества - максимально гибкий, надежный.

Недостатки - на начальном этапе освоения Zabbix, ОС Linux и скриптового программирования, может вызывать сложности.

- Мониторинг баз данных (технология ODBC):

Преимущества – при правильно настроенной связке Zabbix + ODBC это простой в использовании инструмент.

Недостатки – требует компиляции Zabbix с соответствующим пакетом драйвера ODBC на этапе установки Zabbix, присутствуют ограничения самой технологии ODBC на запросы, непрозрачность концепции производимых действий, отсутствует популяризационная информация по внедрению и эксплуатации данного решения.

- SSH - агент (удаленное подключение по SSH к хосту и выполнение на нём скриптов):

Преимущества - можно использовать и команды, и полноценные скрипты. Не нужно использовать дополнительные параметры.

Недостатки - требует компиляции Zabbix с соответствующим пакетом SSH на этапе установки Zabbix, необходимо наличие логина и пароля для доступа по SSH.

- Zabbix-агент (дополнительно установленный на объекте мониторинга демон):

Преимущества - прост в развертывании, не требует сложных настроек.

Недостатки – функционал ограничен выводом статистики СУБД, а так же непрозрачности концепции мониторинга.

Мы будем осуществлять мониторинг базы данных MySQL двумя путями – через Zabbix-агент (уже установлен, прост в использовании), и с помощью внешней проверки (на скриптовом языке Perl Вы создадите свой первый скрипт, извлекающий из базы данных необходимую информацию).

Организуем сбор данных через Zabbix-агент – нужно проверить настройки агента, касающиеся взаимодействий с базой данных.

В файле /etc/zabbix/zabbix_agentd.conf проверьте наличие строки «Include=/etc/zabbix/zabbix_agentd.d/*.conf»:

 # sudo nano /etc/zabbix/zabbix_agentd.conf

Удостоверьтесь, что эта строка есть и раскомментирована (в начале строки стерт символ «#»).

Удостоверьтесь, что файл /etc/zabbix/zabbix_agentd.d/userparameter_mysql.conf существует, и в нем присутствуют переменные:

UserParameter=mysql.status[*],echo "show global status where Variable_name='$1';" | HOME=/var/lib/zabbix mysql -N | awk '{print $$2}'

UserParameter=mysql.size[*],echo "select sum($(case "$3" in both|"") echo "data_length+index_length";; data|index) echo "$3_length";; free) echo "data_free";;

esac)) from information_schema.tables$([["$1" = "all" || ! "$1"]] || echo " where table_schema='$1'")$([["$2" = "all" || ! "$2"]] || echo "and table_name='$2'");" | HOME=/var/lib/zabbix mysql –N

UserParameter=mysql.ping,HOME=/var/lib/zabbix mysqladmin ping | grep -c alive

UserParameter=mysql.version,mysql -V

Обратите внимание на переменную HOME. Создайте каталог, указанный в переменной HOME:

 # sudo mkdir /var/lib/zabbix

В нём создайте файл «.my.cnf» (название с точками, это обязательно) со следующим содержимым:

 # sudo nano /var/lib/zabbix/.my.cnf

[client]

user = agent

password = agent

Этот файл указывает, с какими учетными данными Zabbix-сервер будет подключаться к базе данных. Перезапустите zabbix-agent:

 # sudo service zabbix-agent restart

Удостоверьтесь, что процесс запустился корректно:

 # sudo service zabbix-agent status

Обратите внимание, в файле /var/lib/zabbix/.my.cnf, для доступа к базе данных, использована пара логин/пароль agent/agent. Нужно создать соответствующие учетные данные в базе данных MySQL. Выполните следующие команды (для

администрирования баз данных существует свой собственный, так называемыйструктурированный язык запросов - SQL):

 # mysql –uroot –proot

Осуществлен вход в командную строку управления базы данных.

>CREATE USER 'agent'@'localhost' IDENTIFIED BY 'agent';

Создан пользователь с учетными данными agent/agent.

*>GRANT ALL PRIVILEGES ON *.* TO 'agent'@'localhost';*

Пользователю agent даны полные привилегии на работу с данными базы.

>FLUSH PRIVILEGES;

Изменения применены – без этой команды предыдущие действия не имеют силы.

>quit

Выход из режима конфигурирования базы данных.

Zabbix-агент готов к опросу базы данных. Концепция осуществления мониторинга Zabbix-агентом следующая – Zabbix-агент авторизуется в консоли управления базой данных, и извлекает параметры, запрашиваемые командами «mysqladmin status» и «mysqladmin extended-status». Эти команды также можно вводить в консоли ОС, часть результата приведена на рисунке 8.1.

```
zabbix@zabbix_machine:~$ sudo mysqladmin status -uagent -pagent
mysqladmin: [Warning] Using a password on the command line interface can be inse
cure.
Uptime: 30634  Threads: 63  Questions: 374912  Slow queries: 0  Opens: 692  Flus
h tables: 1  Open tables: 681  Queries per second avg: 12.238
zabbix@zabbix_machine:~$ sudo mysqladmin extended-status -uagent -pagent
mysqladmin: [Warning] Using a password on the command line interface can be inse
cure.
+-------------------------------------------+-------------------------------
-------------------+
| Variable_name                             | Value
                   |
+-------------------------------------------+-------------------------------
-------------------+
| Aborted_clients                           | 1
                   |
| Aborted_connects                          | 0
                   |
| Binlog_cache_disk_use                     | 0
                   |
| Binlog_cache_use                          | 0
                   |
| Binlog_stmt_cache_disk_use                | 0
                   |
```

Рисунок 8.1 – вывод команд «mysqladmin status» и «mysqladmin extended-status»

Файл userparameter_mysql.conf содержит в себе переменные, которые будут извлечены из БД.

Создайте шаблон опроса User Mysql monitor в группе User Templates. В группе узлов сети Agent Monitoring Servers создайте узел сети host.mysql.home (с IP-адресом 127.0.0.1), и присоедините к нему шаблон User Mysql monitor. Приступим к наполнению шаблона элементами данных (Таблица 8.1).

Таблица 8.1 – Элементы данных для мониторинга СУБД MySQL

Элемент данных	Ключ элемента данных	Тип данных, ед-ца изм-я	Примечание
Время работы БД (Инвентаризация)	mysql.status[Uptime]	Числовой	Доступно в меню Инвентаризация
Версия (инвентаризация)	mysql.version	Символ	Доступно в меню Инвентаризация
Статус базы данных	mysql.ping Аналогичен простой проверке icmpping	Числовой	Построить график и создать триггер
Завершено успешно транзакций Commit	mysql.status[Com_commit]	Числовой (с плав. т.), qps, Изменение в секунду	Поместить на один график
Отменено неудачных транзакций RollBack	mysql.status[Com_rollback]	Числовой (с плав. т.), qps, Изменение в секунду	
Операций Update	mysql.status[Com_update]	Числовой (с плав. т.), qps, Изменение в секунду	
Операций Select	mysql.status[Com_select]	Числовой (с плав. т.), qps, Изменение в секунду	
Операций Insert	mysql.status[Com_insert]	Числовой (с плав. т.), qps, Изменение в секунду	
Операций Delete	mysql.status[Com_delete]	Числовой (с плав. т.), qps, Изменение в секунду	
Начато операций	mysql.status[Com_begin]	Числовой (с плав. т.), qps, Изменение в секунду	
Удачных обращений к кэшу	mysql.status[qcache_hits]	Числовой (с плав. т.), qps, Изменение в секунду	
Количество обрабатываемых запросов	mysql.status[Questions]	Числовой, Изменение в секунду	Поместить на один график
Количество медленных запросов	mysql.status[Slow_queries]	Числовой, Изменение в секунду	
Открыто соединений	mysql.status[Threads_connected]	Числовой (с плав. т.)	
Отправлено байт	mysql.status[Bytes_sent]	Числовой (с плав. т.), bps, Изменение в секунду	Поместить на один график
Принято байт	mysql.status[Bytes_received]	Числовой (с плав. т.), bps, Изменение в секунду	

Вы должны получить следующий результат (Рис. 8.2):

Элементы данных

Все шаблоны / User Mysql monitor Группы элементов данных 1 Элементы данных 16 Триггеры 1 Графики 3 Комплексные

Мастер	Имя ▲	Триггеры	Ключ	Интервал	История	Дина
•••	Версия (инвентаризация)		mysql.version	1m	0	
•••	Время работы БД (Инвентаризация)		mysql.status[Uptime]	1m	0	0
•••	Завершено успешно транзакций Commit		mysql.status[Com_commit]	1m	60d	60d
•••	Количество медленных запросов		mysql.status[Slow_queries]	1m	60d	60d
•••	Количество обрабатываемых запросов		mysql.status[Questions]	1m	60d	60d
•••	Операций begin		mysql.status[Com_begin]	1m	60d	60d
•••	Операций delete		mysql.status[Com_delete]	1m	60d	60d
•••	Операций insert		mysql.status[Com_insert]	1m	60d	60d
•••	Операций select		mysql.status[Com_select]	1m	60d	60d
•••	Операций Update		mysql.status[Com_update]	1m	60d	60d
•••	Открыто соединений		mysql.status[Threads_connected]	1m	60d	60d
•••	Отменено неудачных транзакций RollBack		mysql.status[Com_rollback]	1m	60d	60d
•••	Отправлено байт		mysql.status[Bytes_sent]	1m	60d	60d
•••	Принято байт		mysql.status[Bytes_received]	1m	60d	60d
•••	Статус базы данных	Триггеры 1	mysql.ping	1m	60d	60d
•••	Удачных обращений к кэшу		mysql.status[qcache_hits]	1m	60d	60d

Рис. 8.2 - Элементы данных шаблона User Mysql monitor

Триггеры

Все шаблоны / User Mysql monitor Группы элементов данных 1 Элементы данных 16 Триггеры 1 Графики 3

Важность	Имя ▼	Выражение
CPU и Errors	База данных {#HOST.CONN} недоступна	{User Mysql monitor:mysql.ping.last(0)}=0

Рисунок 8.3 – Триггер, срабатывающий на недоступность базы данных

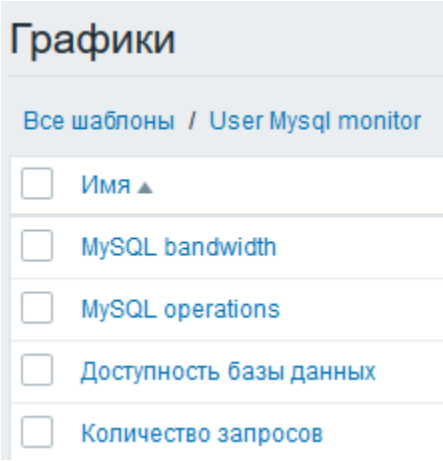

Рисунок 8.4 – Графики шаблона User Mysql monitor

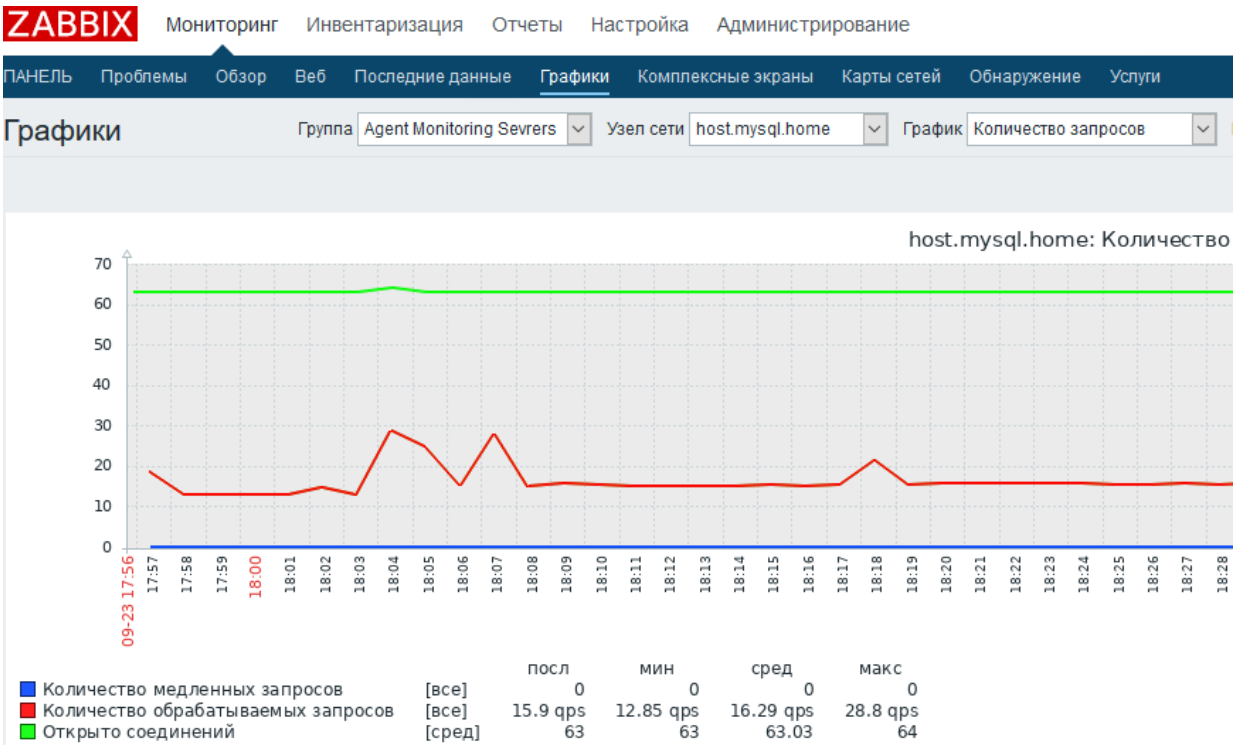

Рисунок 8.5 – График, отражающий количество запросов к БД

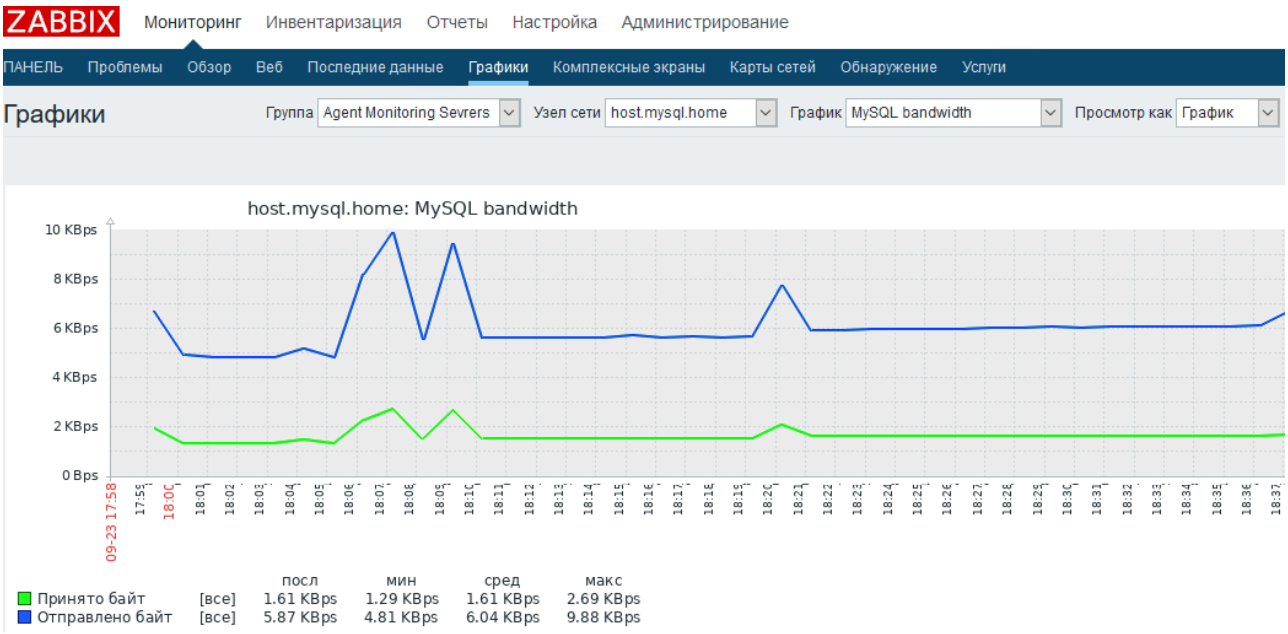

Рисунок 8.6 – График, отражающий объемы трафика, принимаемого и отдаваемого базой данных

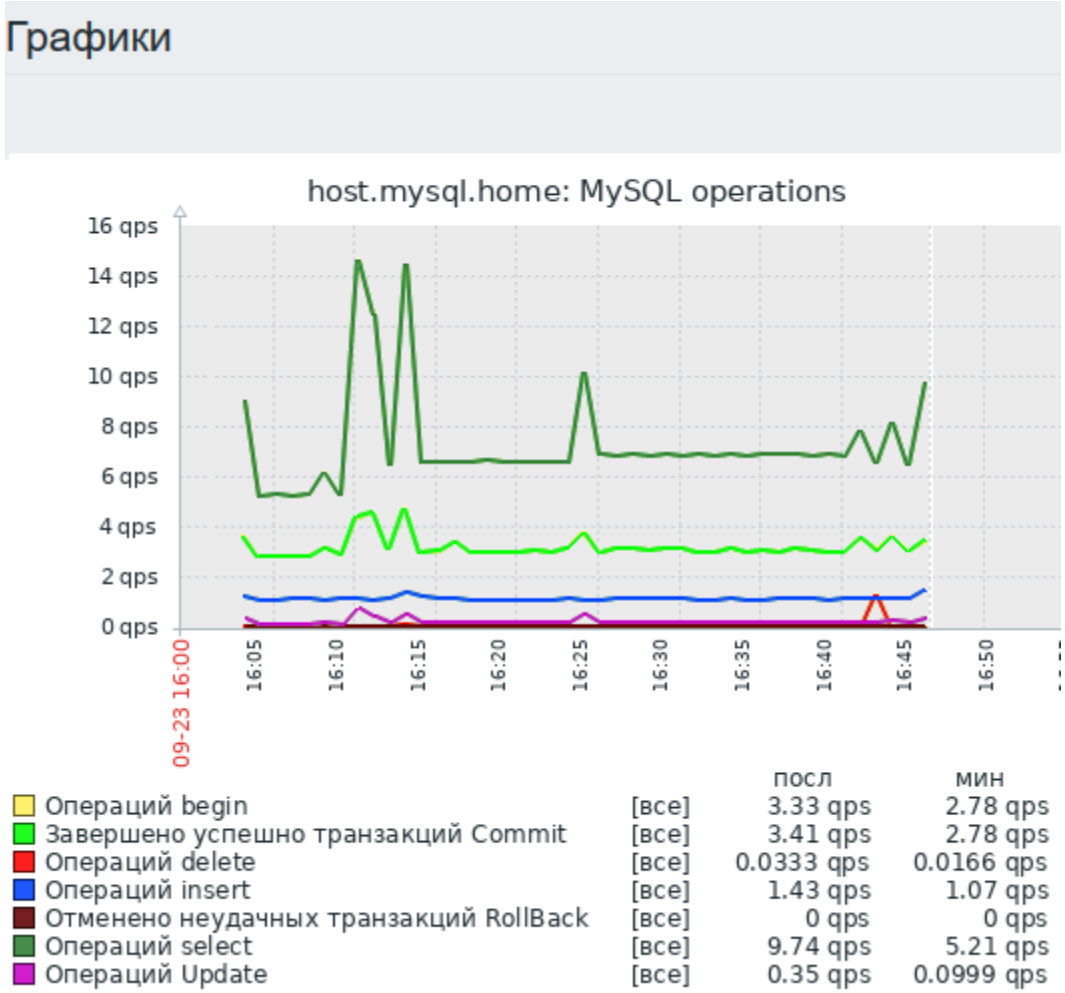

Рисунок 8.7 – График, отражающий количество операций в базе данных

Чтобы понять, что именно отражается на этих графиках, нужно разобраться с принципами работы баз данных.

В источнике [8] дано, на мой взгляд, достаточно приемлемое определение базы данных. База данных - организованная в соответствии с определёнными правилами и поддерживаемая в памяти компьютера совокупность данных, характеризующая актуальное состояние некоторой предметной области и используемая для удовлетворения информационных потребностей пользователей. Далее немного теории.

В нашем случае, сервис Zabbix запрашивает с объектов мониторинга различные параметры и записывает в базу данных, а так же извлекает их оттуда для визуализации по запросам из Zabbix в виде графиков, инвентаризации и прочего. Именно этот объем данных, (принимаемых и отдаваемых базой) отображается на графике 8.6.

Вернемся к рисунку 8.5. Параметр «Открыто соединений» показывает количество подключений сервисов к БД. Посмотреть их можно в консоли MqSQL с помощью команды «SHOW PROCESSLIST;». Там будет отображено порядка 60 процессов для сервиса Zabbix-server, и два для root.

Параметр «количество медленных запросов», или slow query – очень интересный параметр, характеризующий оптимальность работы базы данных. Запрос считается медленным, если база данных не может его обработать в пределах короткого временного промежутка.

По стечению обстоятельств любой запрос может иногда выполняться медленно (ожидание окончания обработки других запросов, занятость дискового пространства сервера, высокая нагрузка на сервер). Случайные медленные запросы не являются признаком проблем. Однако, при систематических долгих запросах возникает большая нагрузка на сервер и базу данных, что приводит к серьезным задержкам или неработоспособности сервиса. Долгое исполнение запроса может быть обусловлено отсутствием индексов (специальных указателей, ускоряющих поиск требуемых данных), либо неоптимально составленными запросами (база данных Zabbix достаточно стабильна, slow query не должны иметь места). При наличии регулярных медленных запросов требуется реорганизация запросов и, возможно, архитектуры базы данных. Задача системы мониторинга – выявить эту проблему.

Параметр «количество обрабатываемых запросов» показывает суммарное количество всех запросов, обрабатываемых базой в данный момент.

Теперь обратимся к рисунку 8.7. SQL - Structured Query Language, или язык сктруктурированных запросов. Основные операторы манипулирования данными (или операторы DML (Data Manipulation Language)), перечислены ниже:

- SELECT - отобрать строки из таблиц;

- INSERT - добавить строки в таблицу;

- UPDATE - изменить строки в таблице;

- DELETE - удалить строки в таблице;

- COMMIT - зафиксировать транзакцию;

- ROLLBACK – отменить транзакцию;

Не вдаваясь в подробности, операторы SELECT, INSERT, UPDATE и DELETE – оперируют непосредственно данными в таблицах. А операторы COMMIT и ROLLBACK – отвечают за транзакции. Транзакция – важное понятие, раскрывающее особенности работы баз данных.

Суть транзакции состоит в объединение нескольких последовательных действий с данными в одну операцию. Внутренние промежуточные состояния между шагами не видны для других транзакций. Если транзакция выполнена успешно, все модификации данных, сделанные в течение транзакции, принимаются и становятся постоянной частью базы данных. Если в результате выполнения транзакции происходят ошибки и должна быть произведена отмена или выполнен откат, все модификации данных будут отменены. Наличие отмененных транзакций является сигналом для администратора базы данных о проблемах с базой данных.

Еще одно понятие - кэш запросов. В нем хранится текст запроса вместе с результатом. При получении другого идентичного запроса сервер может извлечь результаты из кэша запросов, а не выполнять снова тот же самый запрос. Кэш

запросов полезен в архитектурах, где таблицы обновляются не слишком часто и присутствуют идентичные запросы.

Теория функционирования баз данных обширна, это отдельное направление в IT. Более подробную информацию о работе баз данных Вы найдете в продвинутых источниках, таких как [9-10].

Последний элемент данных, который войдет в шаблон опроса User Mysql monitor, будет вычислять объем базы данных. Он будет извлекаться исполняемым скриптом, написанным на скриптовом языке Perl. Скрипт – это небольшая программа, автоматизирующая некоторую задачу, которую пользователь иначе делал бы вручную.

Приступим к написанию скрипта (интерпретатор Perl предустановлен в ОС Ubuntu, расширение файлов perl-скриптов - .pl). Все исполняемые скрипты, которые будет использовать Zabbix, должны находиться в каталоге, указанном в конфигурационном фале /etc/zabbix/zabbix_server.conf, в переменной ExternalScripts. В рамках данного курса, я использовал каталог по умолчанию - /usr/lib/zabbix/externalscripts/.

Создайте файл скрипта:

sudo nano /usr/lib/zabbix/externalscripts/db_size.pl

И внесите в него следующий код:

```
#!/usr/bin/perl

use DBI;

my $host = "localhost"; #MySQL-Server

my $port = "3306"; #Default port MySQL
```

```
my $user = "agent"; #Имя пользователя БД

my $pass = "agent"; #Пароль пользователя

my $db = "zabbix_db"; #Название базы данных

$dbh = DBI->connect("DBI:mysql:$db:$host:$port",

$user,$pass); #Подключение к БД, Enter после $port", обязателен!

$sth = $dbh->prepare("SELECT table_schema $db, sum( data_length + index_length )/1024/1024 FROM information_schema.TABLES GROUP BY table_schema");

$sth->execute; #Исполнение запроса

while ($ref = $sth->fetchrow_arrayref) {

if ("$$ref[1]">50){

print "$$ref[1]";

} #Извлечение параметра из массива полученных данных

}

$rc = $sth->finish;

$rc = $dbh->disconnect; #Закрытие соединения
```

Полностью скопируйте синтаксис и соблюдайте переносы (или их отсутствие), иначе скрипт не будет работать.

Для работы данного скрипта нужен ряд бибилиотек, дополнительно установите пакеты:

#sudo apt-get install libyaml-perl libdbi-perl libdbd-mysql-perl

После написания скрипта, выдайте ему права на исполнение:

sudo chmod +x /usr/lib/zabbix/externalscripts/db_size.pl

Теперь Вы можете проверить работоспособность скрипта, просто вызвав его из консоли ОС - /usr/lib/zabbix/externalscripts/db_size.pl, поскольку скрипт стал исполняемым.

Текст скрипта с цветным форматированием текстового редактора nano приведен на Рисунке 8.8:

```
#!/usr/bin/perl
use DBI;

my $host = "localhost";  #MySQL Server
my $port = "3306"; #Default port MySQL
my $user = "agent";
my $pass = "agent";
my $db = "zabbix_db";

#Connect to MySQL
$dbh = DBI->connect("DBI:mysql:$db:$host:$port",
$user,$pass);
#SQL query
$sth = $dbh->prepare("SELECT table_schema $db, sum( data_length + index_length )/1024/1024  FROM information_schema.TABLES GROUP BY table_schema");
#Query
$sth->execute;

while ($ref = $sth->fetchrow_arrayref) {
if("$$ref[1]">50){

print "$$ref[1]\n";
}
}
$rc = $sth->finish;
$rc = $dbh->disconnect;
Search (to replace):
^G Get Help     ^Y First Line   M-C Case Sens   M-R Regexp      ^N NextHstory
^C Cancel       ^V Last Line    M-B Backwards   ^P PrevHstory   ^R No Replace
```

Рисунок 8.8 – Синтаксис и подсветка скрипта db_size.pl

В данном скрипте была использована библиотека DBI для работы с базами данных, и запрос на языке SQL. Скрипт подключается к базе данных, выполняет запрос, и передает в Zabbix результат. Вы можете выполнить запрос в консоли MySQL, и посмотреть, что он отдает.

Важно! Не копируйте файл с текстом скрипта из ОС Windows в ОС Linux (например, по FTP). В конце каждой строки текстового файла, созданного в

Windows, записывается символ переноса строк«^M».В файлах Linux его очень сложно идентифицировать и удалить (требуется специальная программа). Работать такой скрипт не будет.

Чтобы научиться писать хорошие скрипты, нужно, во-первых, много практиковаться, а во-вторых – изучать используемый язык, его модули и команды. Данный скрипт написан некорректно и неэффективно с точки зрения программирования. Результатом запроса будет таблица данных, состоящая из названий всех доступных баз данных, созданных в MySQL, и их размер. После этого из столбца размеров баз данных извлекается лишь тот, который больше 50 (имеется ввиду размер в Мб). Вам нужно самостоятельно улучшить этот запрос, чтобы он отдавал размер только одной базы данных – zabbix_db, и переписать часть отдачи данных в Zabbix, убрав порог в 50 Мб (подсказка - для этого исправьте запрос к БД, он должен запрашивать table_schema = 'zabbix_db').

Добейтесь работоспособности скрипта, и создайте в шаблоне User Mysql monitor элемент данных «Размер базы данных» с ключом db_size.pl и типом Внешняя проверка (рис. 8.9):

Элементы данных

Все шаблоны / User Mysql monitor Группы элементов данных 1 Элементы данных 17 Триггеры 1 Графики 5 Комплексные эк

Элемент данных Предобработка

* Имя	Размер БД
Тип	Внешняя проверка
* Ключ	db_size.pl Выбрать
Тип информации	Числовой (с плавающей точкой)
Единица измерения	Mb
* Интервал обновления	30s
Пользовательские интервалы	Тип Интервал Период Действие Добавить
* Период хранения истории	60d
* Период хранения динамики изменений	60d
Отображение значения	Как есть показать преобр
Новая группа элементов данных	
Группы элементов данных	-Нет- MySQL
Заполнение поля инвентаря узла сети	Имя

Рисунок 8.9 – Элемент данных db_size.pl, извлекающий размер базы данных

Тип Внешняя проверка означает, что для получения значения будет вызван скрипт, называющийся db_size.pl. Создайте по своему усмотрению триггер и график для этого элемента данных.

Шаблон опроса базы данных готов. Инвентарные данные базы данных должны включать в себя три параметра – версию базы данных, время работы, и размер базы данных (рис. 8.10):

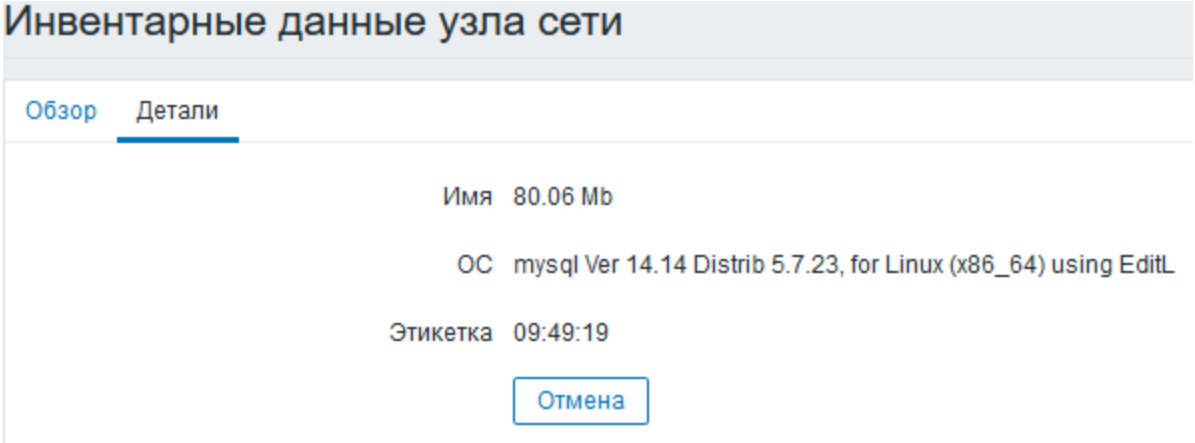

Рисунок 8.10 – Инвентарные данные базы данных

Комплексный экран приведен на рисунке 8.11:

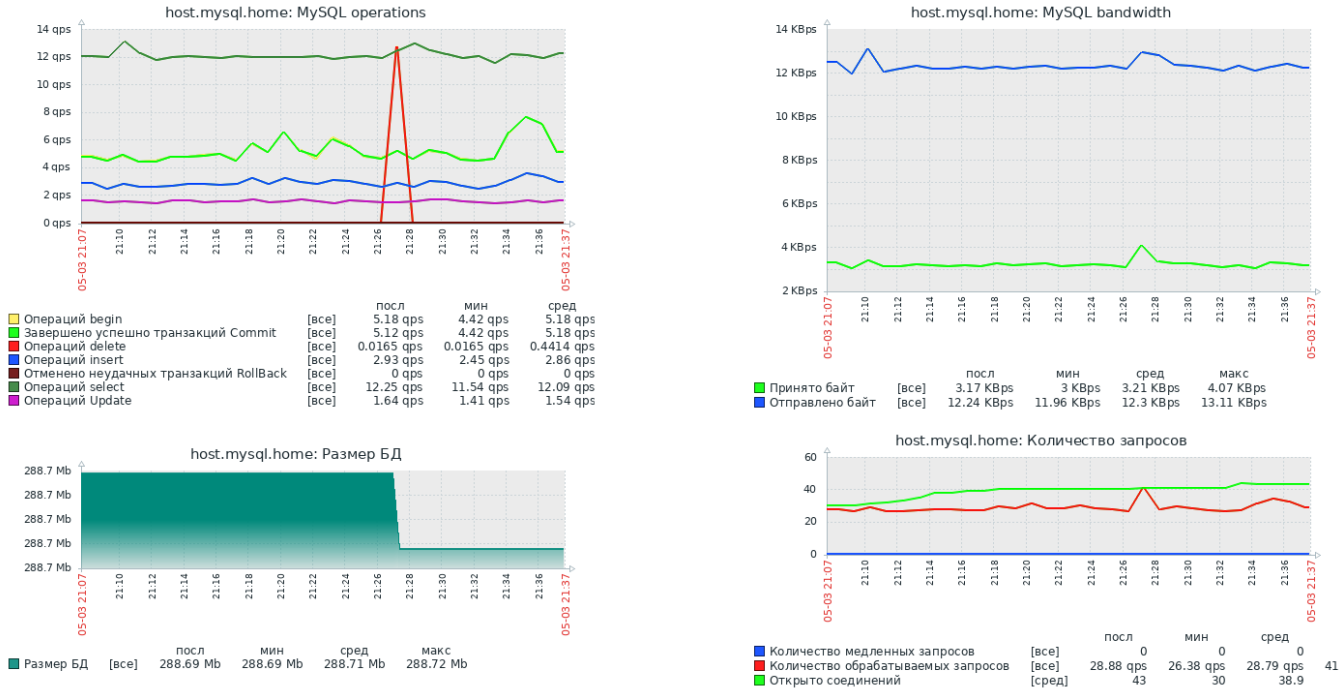

Рисунок 8.11 – Комплексный экран для базы данных

В этом разделе Вы научились осуществлять мониторинг базы данных MySQL (для других баз данных процесс почти не отличается, могут меняться инструменты, но это решается путем изучения документации на БД). Важное достижение – запуск первого исполняемого скрипта. Возможность запускать скрипты в рамках опроса объектов мониторинга является одним из преимуществ Zabbix перед другими системами мониторинга.